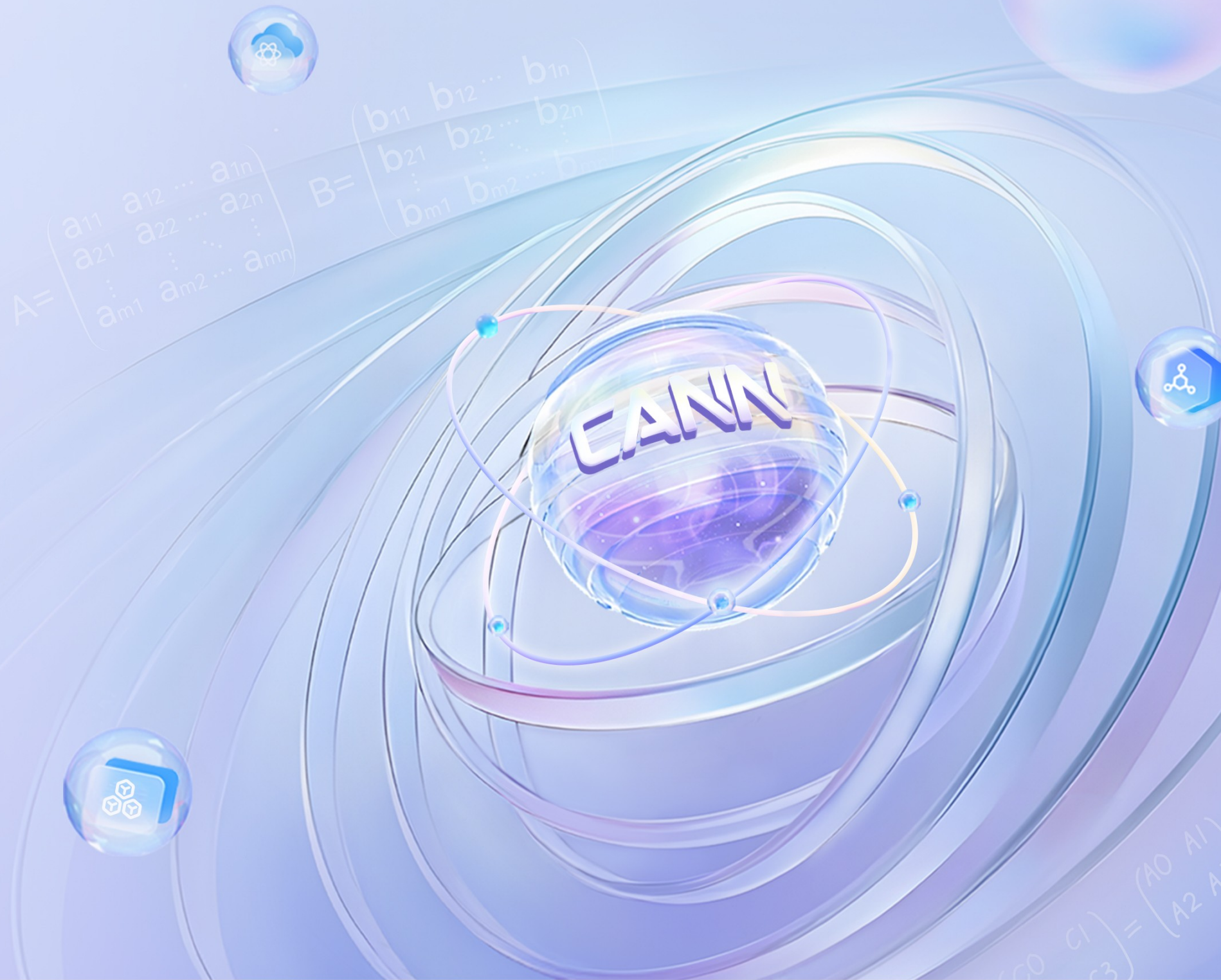


Profiling 与 性能瓶颈定位

昇腾 CANN 启航营

定位性能瓶颈，验证优化成效



目录

- 1 Profiling 基础概念
- 2 如何开启与分析
- 3 Prefill 与 Decode 定位
- 4 教学实践与实验

01 Profiling 基础概念

为什么要 profiling，本节学什么

为什么要做 profiling

profiling 将性能优化从经验驱动转为**数据驱动**，让 " 慢在哪里 " 有据可查。



回答慢在哪里

是 prefill 慢、decode 慢、host 下发慢，还是某个算子特别耗时。



判断资源是否浪费

观察 NPU 是否有空泡、CPU 是否在等待、算子之间是否存在明显间隙。



验证优化是否有效

改配置或改代码后，用同样输入复测，确认指标确实改善。

profiling 为优化决策提供数据依据，是性能工程的基础能力。先定位问题，再实施优化。

性能分析基础定位方法

重点在于**建立方法**，掌握核心工具的使用逻辑，形成系统化的性能定位能力

1

区分两个阶段

Prefill 处理输入上下文； Decode 自回归生成后续 token。

2

关注核心指标

TTFT、TPOT、吞吐；实验中也关注 decode average inference time。

3

构建证据链

现象 → trace 证据 → 瓶颈假设 → 修改 → 复测结果。

4

设计对比实验

以 Qwen3-8B 为例，对比执行模式、输入长度或生成长度。

学习目标

目标是完成一次**可解释的性能分析**，建立系统化的性能调优思维。

核心能力：

- 在 recipes 中开启 profiler
- 区分 prefill 和 decode 的 trace 特征
- 识别空泡、热点和常见瓶颈类型
- 设计单变量实验并撰写证据链报告

02 如何开启与分析

在 recipes 中开启 profiler，掌握标准工作流



在 recipes 中如何开启 profiler

Qwen3-8B 单卡实践中，入口主要是 YAML 配置和 ProfilerManager

YAML 开关

在
models/qwen/config/qwen3_8b_1tp.yaml 中设置：

enable_profiler: True

结果目录

框架自动区分 prefill 与 decode，结果写入：

prof/prefill 与 **prof/decode**

代码入口

executor/utils/profiler_context.py 中的 **ProfilerManager** 管理 profiler 生命周期。

实验建议

先跑 baseline，再开 profiler；每次只改一个变量，方便解释结果。

掌握 profiler 的开启方式后，关键在于理解**何时启用以及如何解读结果**。

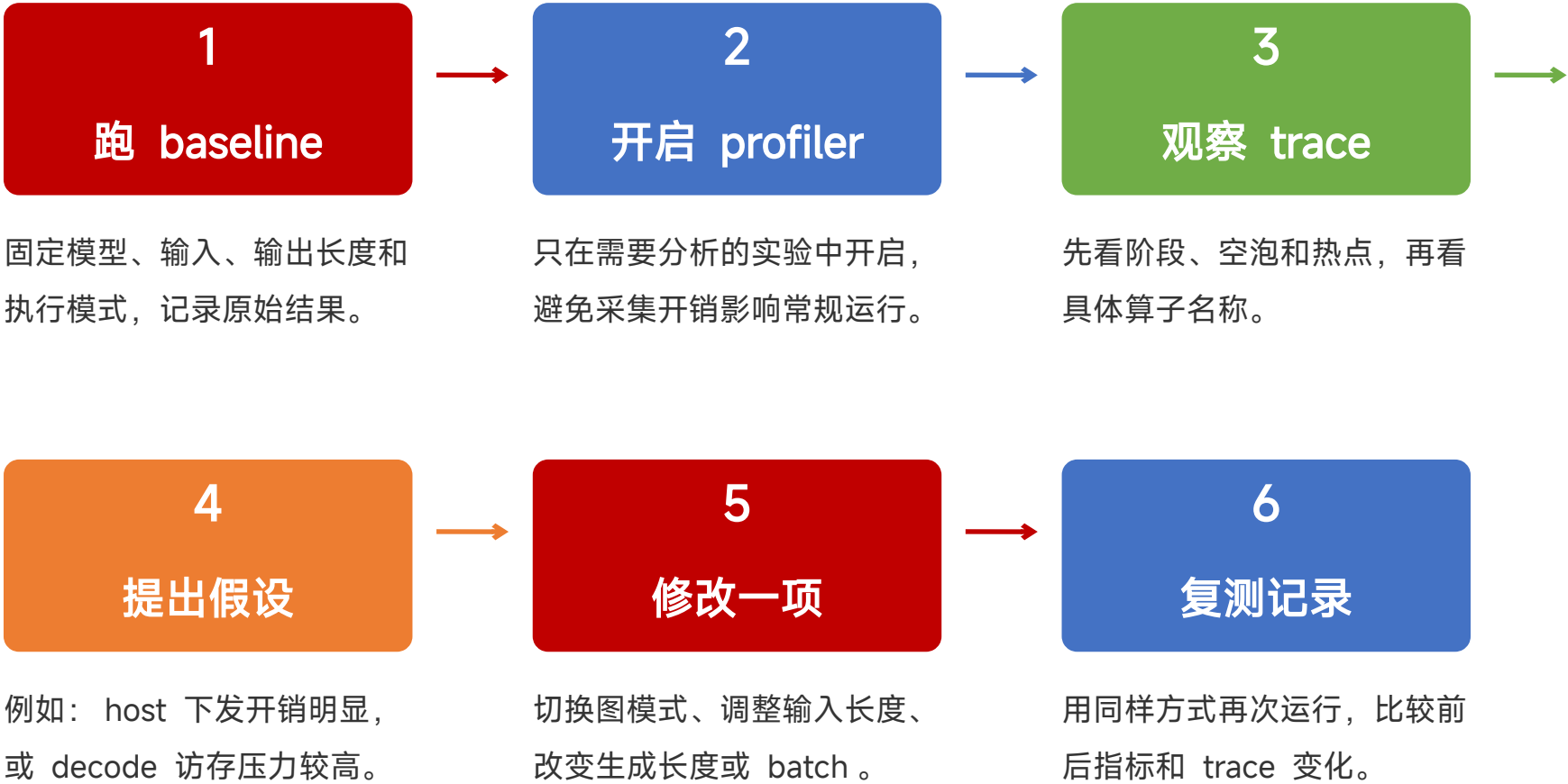
Qwen3-8B Profiler 开关

```
1  model_config:
2    model_name: "qwen3_8b"
3    model_path: "/data/models/Qwen/Qwen3-8B"
4    exe_mode: "ge_graph"
5    enable_profiler: False
6    with_ckpt: True
7    enable_cache_compile: False
8    enable_static_kernel: False
9
10 data_config:
11   dataset: "default"
12   input_truncated_len: 2048
13
14 parallel_config:
15   world_size: 1
16   attn_tp_size: 1
17   moe_tp_size: 1
18   embed_tp_size: 1
19   lmhead_tp_size: 1
20
21 scheduler_config:
22   max_new_tokens: 256
23   batch_size: 1
```

```
1  ∨ model_config:
2    model_name: "qwen3_8b"
3    model_path: "/data/models/Qwen/Qwen3-8B"
4    exe_mode: "ge_graph"
5    enable_profiler: True
6    with_ckpt: True
7    enable_cache_compile: False
8    enable_static_kernel: False
9
10 ∨ data_config:
11   dataset: "default"
12   input_truncated_len: 2048
13
14 ∨ parallel_config:
15   world_size: 1
16   attn_tp_size: 1
17   moe_tp_size: 1
18   embed_tp_size: 1
19   lmhead_tp_size: 1
20
21 ∨ scheduler_config:
22   max_new_tokens: 256
23   batch_size: 1
```

标准工作流：先测量，再优化

将性能分析拆解为固定步骤，以数据为依据推进优化



工作流核心原则

- ① **先稳定，再分析**
稳定的 baseline 是优化结论可信的前提。
- ② **一次只改一项**
单次只修改一个变量，才能准确判断该改动对性能的影响。
- ③ **用数据说话**
每个结论都要有 trace、日志或指标表格支撑。
- ④ **闭环验证**
修改后必须复测，确认假设被支持或推翻。

这六步通常需要多次迭代，在 "观察→假设→修改→复测" 的循环中逐步推进，每次聚焦一个变量。

日志阅读与基线建立

开启 profiler 之前，先从**日志**中获取性能概览

日志中直接看到

- **prefill time** : 输入处理阶段耗时
- **decode step time(s)** : 每步生成耗时
- **average inference time** : 平均推理耗时
- **生成 token 数**: 实际输出长度

换算得到

- $TTFT \approx \text{prefill time} + \text{首步开销}$
- $TPOT \approx \text{decode step time 均值}$
- $TPS = \text{生成 token 数} / \text{总耗时}$
- $\text{Throughput} = \text{batch} \times \text{TPS}$

记录习惯

每次运行记录 **配置 + 日志关键行 + 指标**，形成可对比的性能矩阵，便于后续分析。

日志是**无需额外开销**的性能数据来源。建立先读日志、记录指标的习惯，再决定是否深入 profiling 。

```
4 ModelConfig(model_name='qwen3_8b', model_path='/data/models/Qwen/Qwen3-8B', dtype='bfloat16', platform_version='A3', exe_mode='ge_graph', enable_profiler=False)
DataConfig(dataset='default', input_truncated_len=2048)
ParallelConfig(world_size=1, local_rank=0)
SchedulerConfig(batch_size=1, max_new_tokens=256, max_prefill_tokens=2048)

56 .[INFO][2026-07-10 09:46:37.788][execution_engine.py:390] Warm-up completed successfully.

57 [INFO][2026-07-10 09:46:37.837][scheduler.py:482] [Main] Inference time (prefill): 25.14 ms

58 [INFO][2026-07-10 09:46:37.854][scheduler.py:482] [Main] Inference time (decode): 15.53 ms

59 [INFO][2026-07-10 09:46:37.870][scheduler.py:482] [Main] Inference time (decode): 14.87 ms

60 [INFO][2026-07-10 09:46:37.886][scheduler.py:482] [Main] Inference time (decode): 14.76 ms

314 [INFO][2026-07-10 09:46:41.851][infer.py:77] Request 0: outputs: <think>
315 Okay, so I need to figure out what the output of an attention function is. Let me start by recalling what I know about attention mechanisms. From what I remember, attention
functions are used in models like transformers to help the model focus on relevant parts of the input when processing data.

316
317 The basic idea is that given a query vector and a set of key-value pairs, the attention function computes a weighted sum of the value vectors. The weights are determined by
the similarity between the query and each key. So, the output should be a combination of the values, where each value is multiplied by a weight that depends on how well the
corresponding key matches the query.

318
319 Let me break it down. The query is a single vector, and the keys and values are sets of vectors. For each key in the set, we calculate a score that measures how related the
query is to that key. These scores are then normalized, usually using a softmax function, to form a probability distribution. This distribution is then used to weight the
corresponding values, and the weighted sum of these values gives the output of the attention function.

320
321 Wait, so the output is a vector that's a weighted combination of the values. Each weight is derived from the query and the respective key. So, if
322

323 [INFO][2026-07-10 09:46:41.855][infer.py:105] qwen3_8b decode average inference time cost is 14.74 ms
```

Profiler 输出：三类文件

profiling 结果主要包含四类文件，从聚合统计到逐 kernel 明细，再到时间线，层层递进

文件	主要用途	内容
op_statistic.csv	按 OP type 聚合耗时和调用次数	Top 耗时算子类型，耗时占比（Ratio）
kernel_details.csv	每个 kernel 的明细记录	Duration、Wait Time、Name
trace_view.json	Host 与 NPU 的时间线视图	Host / NPU 任务是否连续，有无空泡

 阅读顺序建议

op_statistic → kernel_details → trace，从宏观到微观逐步深入

三类文件相互补充：op_statistic 定位热点类型，kernel_details 拆解单个算子，trace 验证时间线假设。

op_statistic：Top 耗时算子分析

按 OP 类型聚合统计，快速定位**时间花在哪里**

指标	含义	读法	可能提示
Total Time	某类 OP 的总耗时	按降序排列，看前 5 项	主要时间消耗在哪些算子类型
Count	该类 OP 的调用次数	对比不同 OP 的调用频次	是否存在大量小算子调用
Ratio	耗时占总时间的比例	占比超过 20% 需重点关注	优化的优先级排序依据
Prefill/Decode 对比	两个阶段分别统计	分别查看两个阶段的 Top OP	瓶颈是否与特定阶段相关

提示：Prefill 与 Decode 的 Top OP 类型通常不同，两个阶段需**分别查看、分别分析**

📊 分析技巧

- 先按 Ratio 排序，找出占比最高的算子类型
- 对比 Prefill 和 Decode 的 Top OP，看瓶颈是否集中在某一阶段

🔗 与 kernel_details 联动

- op_statistic 发现某类 OP 占比高后，到 kernel_details 中按 Name 筛选，看具体实例的 Duration 分布

op_statistic 是**聚合视角**，适合快速定位热点类型。发现异常后，再下钻到 kernel_details 和 trace 中验证。

Qwen3-8B decode op_statistics.csv

decode_op.csv — LibreOffice Calc

File Edit View Insert Format Styles Sheet Data Tools Window Help

<

kernel_details：逐 kernel 明细分析

从单个 kernel 的维度拆解耗时构成，定位具体耗时点

关键列	含义	分析方式	异常信号
Name / Type	算子实例名与所属类型	按 Type 分组，看同类 kernel 的分布	某类算子反复出现且数量多
Duration	单次 kernel 的执行耗时	按降序排列，找最长 kernel	少数 kernel 占据了大量时间
Wait Time	任务在队列中的等待时间	Wait 偏大时回到 trace 看间隙	同步或 Host 下发瓶颈的信号

↓ 按 Duration 排序

先找出耗时最长的单个 kernel，确认其 Name 和 Type，判断是否为预期中的算子。

⌚ 关注 Wait Time

Wait 偏大说明任务在排队，回到 trace 中查看对应时间段的间隙来源。

↺ 同类反复出现

同类 kernel 大量出现时，回到 op_statistic 看该类型的总占比和优化空间。

kernel_details 是微观视角，与 op_statistic 的宏观视角形成互补。两者结合，才能对瓶颈形成完整判断。

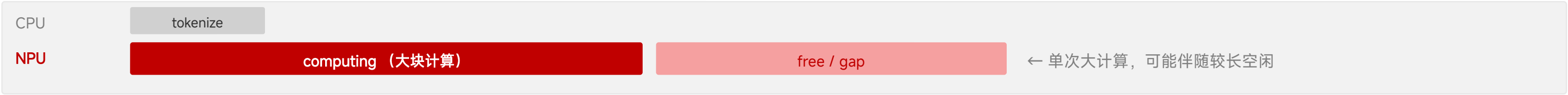
Qwen3-8B decode kernel_details.csv

decode_kernel.csv — LibreOffice Calc												
File Edit View Insert Format Styles Sheet Data Tools Window Help												
Liberation Sans 10 pt B I U A % 0.0 0.0 0.0												
A1 fx Σ = Step Id												
	A	B	C	D	E	F	G	H	I	J	K	
1	Step Id	Device_id	Model ID	Task ID	Stream ID	Name	Type	OP State	Accelerator Core	Start Time(us)	Duration(us)	V
2	3	0	4294967295	1838	47	acInnInplaceOne_OnesLikeAiCore_OnesLike	OnesLike	dynamic	AI_VECTOR_COPY	1783648087743234.852	10.42	
3	3	0	4294967295	1839	47	acInnTril_Tril_Tril	Tril	dynamic	AI_VECTOR_COPY	1783648087743247.532	26.541	
4	3	0	4294967295	1840	47	acInnBitwiseNot_LogicalNotAiCore_LogicalNot	LogicalNot	dynamic	AI_VECTOR_COPY	1783648087743274.093	10.92	
5	3	0	4294967295	1841	47	acInnAdds_AddAiCore_Add	Add	dynamic	AI_VECTOR_COPY	1783648087743408.396	2.5	
6	3	0	4294967295	1843	47	acInnAdds_AddAiCore_Add	Add	dynamic	AI_VECTOR_COPY	1783648087743470.017	2.02	
7	3	0	4294967295	1844	47	acInnInplaceFillScalar_FillAiCore_Fill	Fill	dynamic	AI_VECTOR_COPY	1783648087743495.297	1.46	
8	3	0	4294967295	1845	47	acInnCumsum_CumsumAiCpu_Cumsum	Cumsum	dynamic	AI_CPU	1783648087743554.779	75.361	
9	3	0	4294967295	1846	47	acInnCumsum_CumsumAiCpu_Cumsum	Cumsum	dynamic	AI_CPU	1783648087743630.180	41.301	
10	3	0	4294967295	1847	47	acInnInplaceZero_ZerosLikeAiCore_ZerosLike	ZerosLike	dynamic	AI_VECTOR_COPY	1783648087743671.501	1.3	
11	3	0	4294967295	1849	47	acInnFloorDivides_FloorDivAiCpu_FloorDiv	FloorDiv	dynamic	AI_CPU	1783648087743939.566	78.242	
12	3	0	4294967295	1850	47	acInnRemainderTensorScalar_BroadcastToAiCore_BroadcastTo	BroadcastTo	dynamic	AI_VECTOR_COPY	1783648087744017.828	1.7	
13	3	0	4294967295	1851	47	acInnRemainderTensorScalar_FloorModAiCore_FloorMod	FloorMod	dynamic	AI_VECTOR_COPY	1783648087744019.548	2.26	
14	3	0	4294967295	1852	47	acInnMax_ReduceMaxAiCore_ReduceMax	ReduceMax	dynamic	MIX_AIV	1783648087744022.828	2.9	
15	3	0	4294967295	1853	47	acInnGather_CastAiCore_Cast	Cast	dynamic	AI_VECTOR_COPY	1783648087744122.590	1.66	
16	3	0	4294967295	1854	47	acInnGather_GatherElementsV2_GatherElementsV2	GatherElementsV2	dynamic	AI_VECTOR_COPY	1783648087744124.450	4.28	
17	3	0	4294967295	1855	47	acInnMuls_MulAiCore_Mul	Mul	dynamic	AI_VECTOR_COPY	1783648087744136.230	1.5	
18	3	0	4294967295	1856	47	acInnAdd_CastAiCore_Cast	Cast	dynamic	AI_VECTOR_COPY	1783648087744164.991	1.2	
19	3	0	4294967295	1857	47	acInnAdd_AddAiCore_Add	Add	dynamic	AI_VECTOR_COPY	1783648087744166.791	2.08	
20	3	0		2	1860	47 UpdateModelParam_static_bin	Data	dynamic	AI_VECTOR_COPY	1783648087745365.195	8.32	
21	3	0		2	0	44 GatherV2	GatherV2	static	AI_VECTOR_COPY	1783648087745402.855	7.441	
22	3	0		2	1	44 IndexByTensor/Index	Index	static	AI_VECTOR_COPY	1783648087745410.336	6.06	
23	3	0		2	2	44 IndexByTensor_1/Index	Index	static	AI_VECTOR_COPY	1783648087745416.436	2.38	
24	3	0		2	3	44 RmsNorm	RmsNorm	static	AI_VECTOR_COPY	1783648087745418.856	4.36	
25	3	0		2	4	44 MatMul	MatMul	static	AI_CORE	1783648087745423.256	48.961	
26	3	0		2	5	44 RmsNorm_1	RmsNorm	static	AI_VECTOR_COPY	1783648087745472.257	11.3	
27	3	0		2	6	44 RmsNorm_2	RmsNorm	static	AI_VECTOR_COPY	1783648087745483.577	6.64	
28	3	0		2	7	44 ApplyRotaryPosEmb	ApplyRotaryPosEmb	static	AI_VECTOR_COPY	1783648087745490.237	3.14	
29	3	0		2	8	44 ScatterNdUpdate	ScatterNdUpdate	static	AI_VECTOR_COPY	1783648087745493.397	8	
30	3	0		2	9	44 ScatterNdUpdate_1	ScatterNdUpdate	static	AI_VECTOR_COPY	1783648087745501.417	2.621	
31	3	0		2	12	44 FusedInferAttentionScore	FusedInferAttentionScore	static	MIX_AIC	1783648087745505.258	20.46	
32	3	0		2	13	44 MatMul_1	MatMul	static	AI_CORE	1783648087745526.558	35.021	
33	3	0		2	14	44 InplaceAddRmsNorm	InplaceAddRmsNorm	static	AI_VECTOR_COPY	1783648087745561.599	2.6	
34	3	0		2	15	44 MatMul_2	MatMul	static	AI_CORE	1783648087745564.219	174.123	
35	3	0		2	16	44 StridedSliceV2	StridedSliceD	static	AI_VECTOR_COPY	1783648087745738.382	1.9	
36	3	0		2	17	44 StridedSliceV2_1	StridedSliceD	static	AI_VECTOR_COPY	1783648087745740.302	1.78	
37	3	0		2	18	44 SwishMul	AutomaticBufferFusionOp	static	AI_VECTOR_COPY	1783648087745742.102	3.2	
38	3	0		2	19	44 MatMul_3	MatMul	static	AI_CORE	1783648087745745.322	84.162	
39	3	0		2	20	44 InplaceAddRmsNorm_1	InplaceAddRmsNorm	static	AI_VECTOR_COPY	1783648087745829.524	2.96	

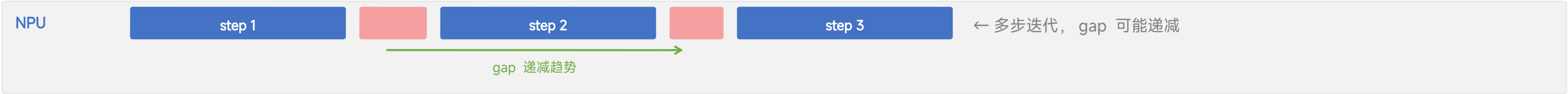
trace_view：阶段识别与空泡定位

先识别**阶段结构**，再定位空泡与热点； Prefill 和 Decode 的 Trace 需分别查看

Prefill 典型模式



Decode 典型模式



■ Prefill Computing ■ Decode Computing ■ Free / Gap ■ Host 侧

阶段识别

Prefill 为单次大块计算，Decode 为多步迭代生成。
先确认阶段归属，再选择分析策略。

空泡定位

Free / Gap 时间可能来自 Host 等待、数据准备、同步或调度间隙。
空泡是资源浪费的信号，需定位来源。

热点发现

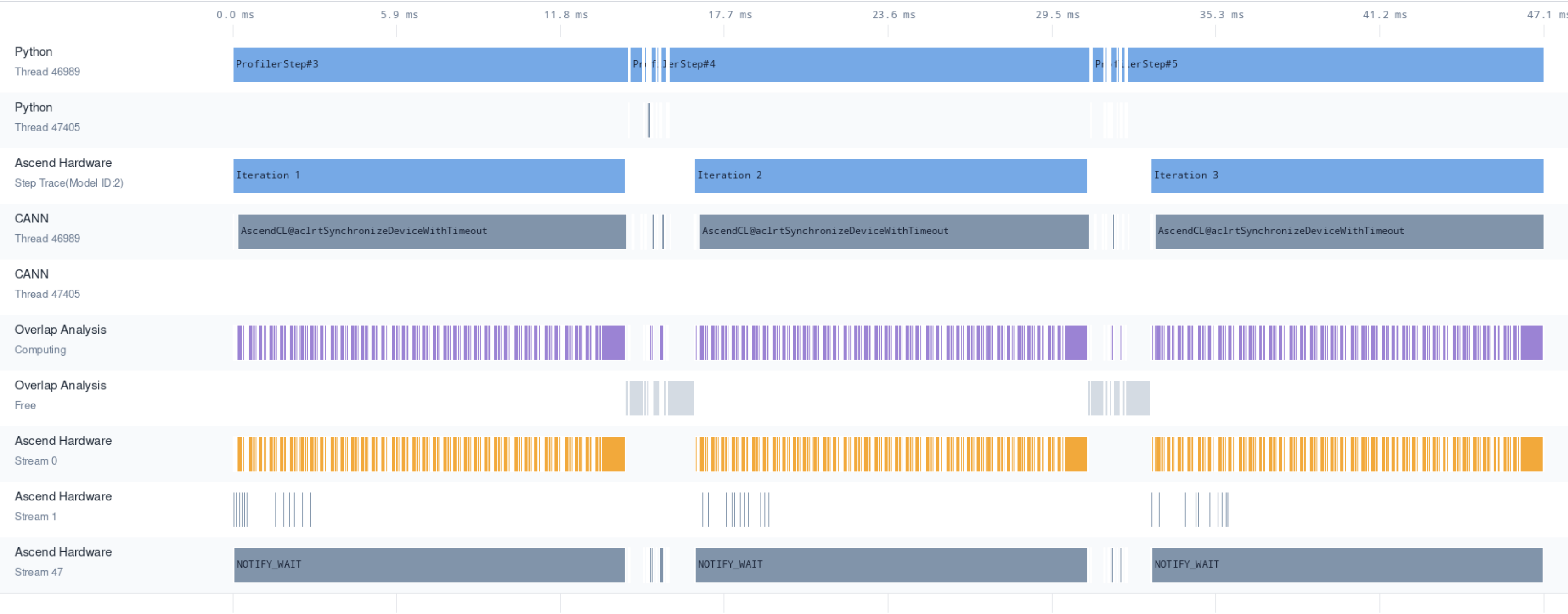
最长、最密集的算子区域优先优化。
热点与 op_statistic 的 Top OP 相互印证。

分析 Trace 时**从宏观到微观**：先确认阶段 → 定位空泡 → 聚焦热点。与 op_statistic、kernel_details 交叉验证。

Qwen3-8B decode trace_view.json

trace_view.json

DECODE



03 Prefill 与 Decode 定位

两个阶段的瓶颈可能完全不同



Prefill 与 Decode 的定位重点不同

Prefill 与 Decode 阶段的性能特征及瓶颈类型存在**显著差异**

Prefill

输入处理阶段

特征：处理完整 prompt，输入长，计算块大，矩阵和 attention 计算集中。

关注重点：

- MatMul / Attention 类算子热点
- 输入长度对 TTFT 的影响
- 总体吞吐和算力利用程度

瓶颈倾向：计算密集型，长输入时矩阵乘法开销大

Decode

自回归生成阶段

特征：每步新增一个 token，步长短，重复次数多，易受访存和调度影响。

关注重点：

- 每步耗时均值与波动
- 调度开销和小算子累积
- KV Cache 读取与空泡

瓶颈倾向：访存密集 + 调度开销，步数多时被放大

实验设计原则：改变输入长度主要影响 **prefill**；改变 max_new_tokens 主要拉长 **decode** 观察窗口。分析时先分阶段，再定位。

常见性能瓶颈的识别与定位

用现象→证据→下一步的链条快速定位问题方向

现象	证据（从 profiling 中看到）	下一步
prefill 慢	MatMul / Attention 类算子在 op_statistic 中占比高；prefill trace 中 computing 块长	检查融合算子是否生效、是否使用图模式、输入 shape 是否合理
decode 慢	decode step 均值高或波动大；kernel_details 中同类算子反复出现	检查小算子调度开销、KV Cache 访存、stream 同步方式
Wait 高 / 空泡多	trace 中出现明显间隙；kernel_details 中 Wait Time 偏大	检查 Host 下发速度、CPU/NPU 同步点、stream 配置
结果波动大	同配置多次运行指标差异明显；trace 结构不稳定	固定 warmup 轮次、排除编译干扰、确认环境一致性

🔍 排查原则

先归类现象，再查证据，最后确定下一步。

🔗 工具联动

op_statistic → kernel_details → trace，三者交叉验证。

遇到性能问题时，**先归类、再验证、后行动**。不要跳过证据直接调参。



04

教学实践与实验

以 Qwen3-8B 为例完成一次可解释的性能分析

Qwen3-8B 教学实践：三组可控变量

控制**单一变量**，确保实验结果可解释

Mode · 执行模式

eager 与 ge_graph 对比

观察：图模式是否减少调度开销，trace 中空泡是否减少。

Prompt · 输入长度

短 prompt 与长 prompt 对比

观察：prefill 阶段耗时变化，TTFT 与输入长度的关系。

Decode · 生成长度

max_new_tokens 调整

观察：decode 重复步数和平均 decode 耗时，TPOT 是否稳定。

记录方式

每组实验保留 YAML 配置、运行命令、日志关键行和 profiling 目录，确保结果可复现。

三组实验覆盖了三类最常见的**性能变量**。建议每组至少跑 3 次取平均，排除随机波动干扰。

学生实验任务

目标是完成一次**最小可行**的性能分析报告

1 跑 baseline

使用 Qwen3-8B 单卡配置跑一次，记录模型、输入和输出长度。

2 开启 profiler

开启 enable_profiler，重新运行并找到 prefill / decode profiling 结果。

3 观察与假设

观察 trace，写出一个瓶颈假设，例如 decode 阶段存在明显间隙。

4 修改一项

只修改一个变量，例如切换执行模式或调整 prompt 长度。

5 复测比较

复测并比较结果，用一张表说明前后变化。

6 写结论

按照 "现象 - 证据 - 假设 - 修改 - 结果" 的结构撰写分析结论。

六步中**第 3 步（观察与假设）**最为关键，从 trace 中准确识别问题，是后续步骤有效性的基础。

实验报告模板

评价重点在于证据链的完整性与清晰度，而非优化幅度

A 现象

说明 baseline 中表现不理想的阶段或指标，给出具体数值。

B 证据

描述 profiling 中的关键发现：热点、空泡、重复算子或编译开销。

C 假设

提出瓶颈假设，说明依据，引用 trace 中的具体位置。

D 修改

明确修改的单一变量：执行模式、输入长度或生成长度。

E 结果

对比复测前后的指标变化，用数据验证假设。

F 反思

说明后续优化方向和当前分析中的不确定性。

好的报告不在于篇幅，而在于 **A→B→C→D→E 形成完整链条**：现象 → 证据 → 假设 → 修改 → 结果。

从 profiling 到高质量 Issue

好的性能报告包含**五个要素**，确保可复现、可判断、可协作

要素	应包含	目的
环境	CANN 版本、torch_npu 版本、Atlas 设备型号	保证他人可复现相同环境
配置	YAML 关键项、batch_size、prompt、执行模式	排除变量干扰，聚焦问题本身
现象	日志关键行、TTFT / TPOT / TPS 数值	说明问题是什么
证据	Profiler 关键截图或 CSV 数据、trace 特征	支持判断，避免主观描述



期望

想优化什么、期望达到什么指标——便于协作者理解目标，给出针对性建议。

日志、CSV、trace 和结论要**一一对应**。证据链完整，Issue 才有人愿意跟进。

小结：用 profiling 构建证据驱动和优化流程

先建立 baseline

稳定基线是判断优化有效性的前提。每次实验前固定模型、输入和配置。

先看大结构

阶段、空泡、热点比单个 kernel 名称更适合初学者入门。先抓大放小。

一次只改一项

控制变量能让结论更清晰，也便于在实验报告中准确归因。

回到开源实践

清晰的问题复现、日志和 profiling 证据，本身就是高质量 Issue 的基础。

性能优化的每个结论都应以为 trace、日志或实验数据为支撑。

profiling 的核心价值在于将优化决策从主观判断转变为**数据驱动**。

Thank you.

社区愿景：打造开放易用、技术领先的 AI 算力新生态

社区使命：使能开发者基于 CANN 社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的 CANN 生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上 CANN 社区获取干货



关注 CANN 公众号获取资讯